CSE207 – Data Structures
Project: Queue Simulator

Project Description:

An important application of queue is queue simulator, a modeling activity used to generate statistics about performance of queue. For this you have to build a single-server single queue simulator. For example, you have to build this simulator for a super shop. This shop has one window, and a clerk can serve one customer at a time. The simulator needs to check three events; the arrival of customer, the start of customer's processing and the completion of customer's processing. For constructing queue simulator you have to do the following:

a. Use queue for constructing customer's queue
b. For the server each call history is saved in a stack
c. Use separate structure for current customer status and simulation statistics.

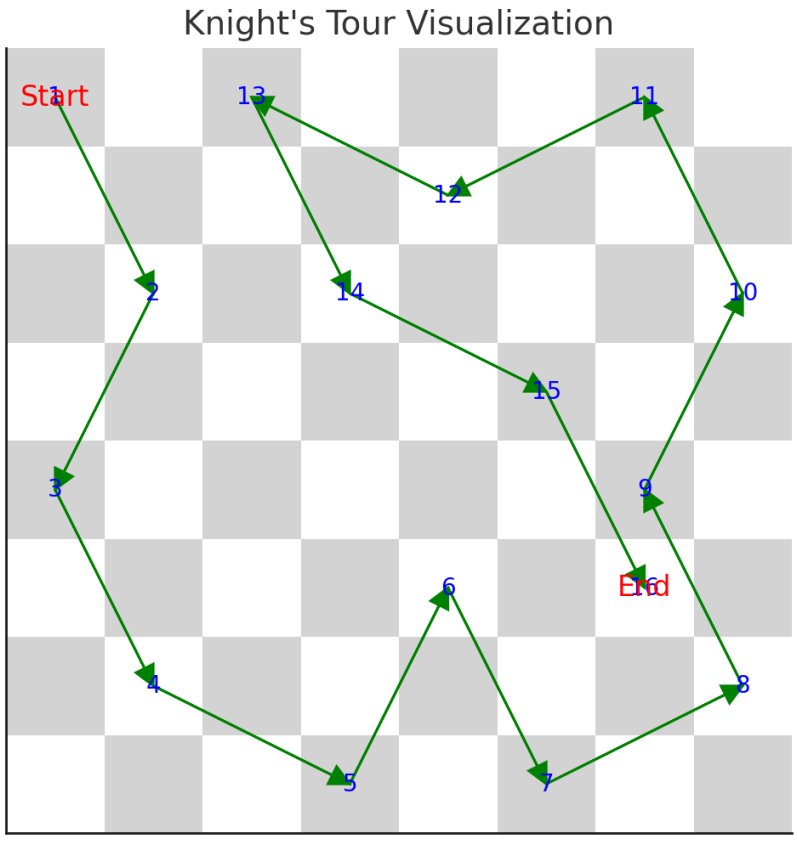
In this project you have to do the following:

1. Print statistics gathered per day during the simulation along with the average queue wait time and average queue service time
2. Show the arrival time, start time, wait time, service time and the number of elements in the queue, each time a customer is completely served.

CSE207 – Data Structures
Project: Knight's Tour

Project Description:

A knight starts at a given position on an N x N chessboard. The goal is to find a path where the knight visits every cell exactly once, following the rules of a knight's movement in chess.



- **Rules**:
  - The knight moves in an L-shape: 2 squares in one direction and 1 square perpendicular to it.
  - Some cells may be blocked (indicated as 0 in the matrix), and the knight cannot land on these.

- **Requirements**:
  - Use recursion or backtracking to find a solution.
  - Represent the board using a matrix.
  - Highlight the path taken by the knight.

CSE207 – Data Structures
Project: Mini Parser

Project Description:

A parser is a software component that takes input data (frequently text) and builds a data structure – often some kind of parse tree, abstract syntax tree or other hierarchical structure – giving a structural representation of the input, checking for correct syntax in the process. In this project, you have to build a small parser that will parse branching statement (*if and if-else-else if*) and algebraic expression of a source program and check for correct syntax in the program.

For example consider the below code.

```
1. int main()
2. {
3. int  a, b,c;
4.  a= 3; b=2;
5.  if[d>b)
6.  c = (a+b)/a;
7.  else
8.  c = (a+b/b;
9.  return 0;
10. }
```

Here we have found three errors. They are: **if [d>b)** in line no 5**, c = (a+b/b** in line no 8 **and d is an undefined variable** in line no 5. So, in this project you have to build a parser that will check syntactical error of a source code. To construct the **syntax parser** you may consider the following data structure,

a. A file that contains a small program containing mathematical expression and branching statements.
b. Stack for checking correct syntactic structure of given source code

In this project you have to do the following:

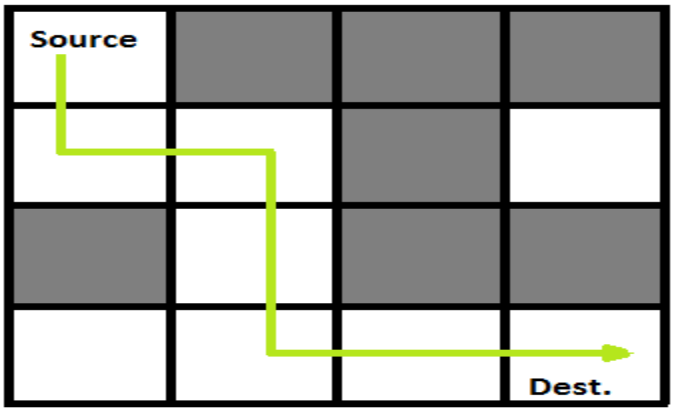
1. Build a parser that will parse algebraic expression
2. Parse structure of branching statement

If error found then provide an error message with line number of the source program

CSE207 – Data Structures
Project: Rat in a Maze

Project Description:

Rat in a maze is a popular puzzle game where a mouse start from a start point to final point by visiting different internal spots. To get to his final destination the mouse need to visit one point two or three times or need to backtrack.



A Maze is given as N*N binary matrix of blocks where source block is the upper left most block i.e., maze [0][0] of the above figure and destination block is lower rightmost block i.e., maze[N-1][N-1]. A rat starts from source and has to reach destination. The rat can move in four directions: right, left, top and down. In the maze matrix, 0 means the block is dead end and 1 means the block can be used in the path from source to destination. To develop the game you may use the following data structure:

a. Matrix for the representation of the Maze
b. Stack or graph to figure out the movement of the rat where a rate cannot move towards the dead block.

In this project you have to do the following:

1. Simulate a mouse movement through the maze
2. Show the path through the maze moved by the mouse

CSE207 – Data Structures
Project: Postfix and Prefix Calculator

Project Description:

You have been using one of very classic calculator, the HP-35, for a long time. It was the first handheld calculator manufactured by Hewlett Packard in 1972. However, after a disastrous accident (dropped it in a sink), it is no longer functional. You miss this calculator so much You finally decided to implement its special form of postfix calculation yourself. For constructing the postfix calculator you have to do the following:

a. Use stack for converting infix expression to postfix and prefix expression
b. Use stack for calculating value of postfix expression
c. Use queue to calculate value of prefix expression

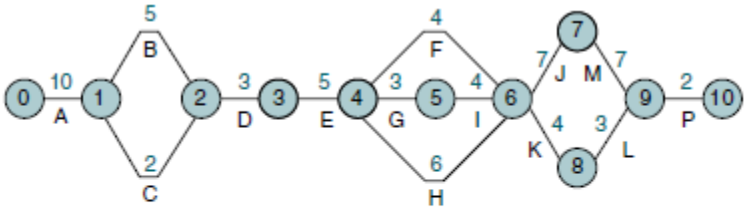
In this project you have to do the following:

1. Take infix expression as input
2. Convert postfix and prefix expression
3. Evaluate value of postfix and prefix expression

CSE207 – Data Structures
Project: CPM Manager

Project Description:

One of the tools used to manage large projects is known as the critical path method (CPM). In CPM the manager builds a network of all phases of a project and then evaluates the network to determine critical aspects of the project. In a CPM network, each vertex is an event, such as the start or completion of a task. The arcs connecting the vertices represent the duration of the activity. Unlike the examples in the text, they also store the name of the activity. To better understand the concept, let's look at a possible CPM plan to build a house. The network for this project is shown in below figure:



A. Prepare plans (10)
B. Building permits (5)
C. Select contractor (2)
D. Foundation (3)
E. Walls (5)
F. Electrical (4)
G. Roof (3)
H. Plumbing (6)
I. Drywall (4)
J. Paint outside (7)
K. Paint inside (4)
L. Fixtures/carpet (3)
M. Landscape (7)
P. Final inspection (2)

In the plan we see that it will take 10 days to prepare the building plan (A) and 5 days to get it approved (B). Furthermore, we can't start building until we have selected the contractor (C). Our plan is to construct the shortest path from the start to the end for our plan. For constructing the shortest path you have to do the following:

a. Build a graph for the network
b. Build Minimum spanning tree from the graph to find shortest possible completion path

In the project you have to do the following:

1. What is the shortest possible completion time (SPCT)? The SPCT is the shortest path through the graph from beginning to end.
2. What is the earliest start time (EST) for each activity? The EST is the sum of the weights in the minimum spanning tree up to the activity.
3. What is the latest start time (LST) for each activity? The LST is the SPCT for the whole project minus the SPCT for the rest of the project
4. What is the critical path for the project? (The critical path is the subgraph consisting of the minimum spanning tree.)

CSE207 – Data Structures
Project: Resolving Overbooked Flight

Project Description:

An airline company uses the formula shown below to determine the priority of passengers on the waiting list for overbooked flights. Priority number = A / 1000 + B – C where,

A is the customer's total mileage in the past year
B is the number of years in his or her frequent flier program
C is a sequence number representing the customer's arrival position when he or she booked the flight.
Given a file of overbooked customers as shown in below Table, write a program that reads the file and determines each customer's priority number. You have to build an application for the airline company that builds a priority queue using the priority number and prints a list of waiting customers in priority sequence.

Name	Mileage	Years	Sequence
Bryan Devaux	53,000	5	1
Amanda Trapp	89,000	3	2
Baclan Nguyen	93,000	3	3
Sarah Gilley	17,000	1	4
Warren Rexroad	72,000	7	5
Jorge Gonzales	65,000	2	6
Paula Hung	34,000	3	7
Lou Mason	21,000	6	8
Steve Chu	42,000	4	9
Dave Lightfoot	63,000	3	10
Joanne Brown	33,000	2	11

For constructing the application you have to do the following:

a. Build a priority queue(binary heap) based on the priority number
b. Build a stack to store information of customer who will get flight ticket later

In the project you have to do the following:

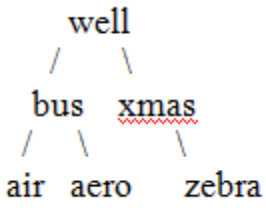
1. Builds a priority queue using the priority number
2. Prints a list of waiting customers in priority sequence.
3. Print details of customer who took flight ticket later

CSE207 – Data Structures
Project: Word Dictionary

Project Description:

Dictionary means a reference book listing alphabetically terms or names important to a particular subject or activity along with discussion of their meanings and applications. Dictionary should have different functionalities such as add new word, search word, delete word and find similar meaning word. For constructing dictionary you have to do the following:

a. Use Binary Search Tree to store the list of 50 words.
b. Each node of BST stores a key of a dictionary. Key 'k' of a node is always greater than the keys present in its left sub tree. Similarly, key 'k' of a node is always lesser than the keys present in its right sub tree.
   Example:



In the above example, keys present at the left sub-tree of the root node are lesser than the key of the root node. And also, the keys present at the right sub-tree are greater than the key of the root node.

In this project you have to do the following,

1) Add new word, search word, delete word and find similar meaning word.
2) All the data may input from a text file where each entry include a word and its definition(meaning)
3) When you search for a particular word then it will give you suggestion of nearest words.
   Example: if you search for "Carn"
   Output: the word "carn" was not found,
   Did u mean "cart", " card ", " carrot", "carrom"

CSE207 – Data Structures
Project: Call distributor in Call center

Project Description:

A call center is a centralized office used for receiving or transmitting a large volume of requests by telephone. This project will contain multiple servers that will solve different problem over phone and one queue that will hold call of customer. Each of the servers can attend call of customer for 10 min. After finishing the call, another call is given to the free server. For constructing virtual call center you have to do the following:

a. Use linked list for constructing multiple server
b. For each server each call history is saved in a stack
c. Customer call will store in a single queue

In this project you have to do the following:

1. Forward call to different server
2. Count total call received by per server during operating system
3. Total time per customer waiting for their calls

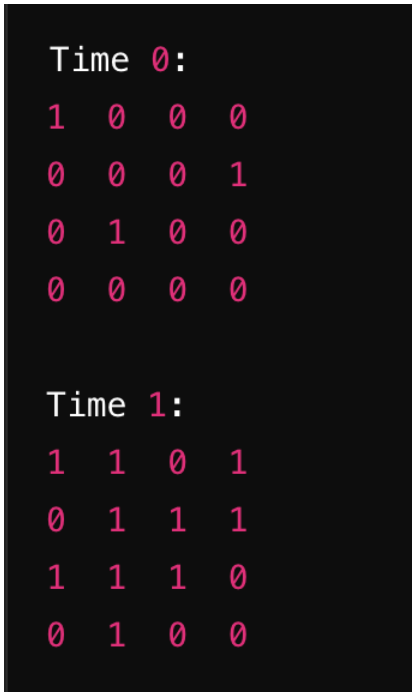
CSE207 – Data Structures
Project: Zombie Outbreak Simulation

Project Description:

A zombie outbreak starts in a city represented as an N x N grid. Some cells are safe zones (indicated by 0), and some are infected zones (indicated by 1). At each time step, a zombie spreads to adjacent cells. The goal is to simulate the spread of the infection and calculate how many time steps it takes to infect the entire city.

Rules:
- Use a queue to simulate the infection spread (Breadth-First Search).
- The infection spreads up, down, left, and right.

Requirements:
- Represent the city using a matrix.
- Output the number of time steps required to infect all reachable zones.

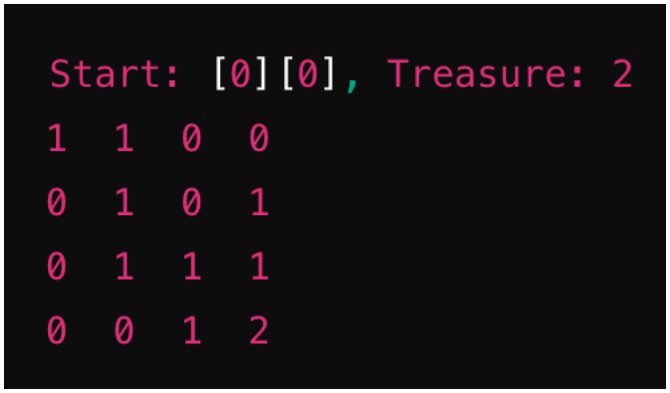


CSE207 – Data Structures
Project: Treasure Hunt

Project Description:

A pirate needs to navigate a grid to find a hidden treasure. The grid contains traps (indicated by 0), open paths (indicated by 1), and the treasure (indicated by 2). The pirate starts at the top-left corner and must reach the treasure in the fewest steps.

- **Rules**:
  - The pirate can move up, down, left, or right.
  - Use Breadth-First Search (BFS) to find the shortest path.
- **Requirements**:
  - Simulate the pirate's movement using a queue.
  - Display the path taken by the pirate to reach the treasure.



CSE207 – Data Structures

Project: Traffic Flow Manager

Project Description:

Design a system to simulate and optimize traffic flow at a city's intersections.
The city is modeled as a **directed weighted graph**, where:

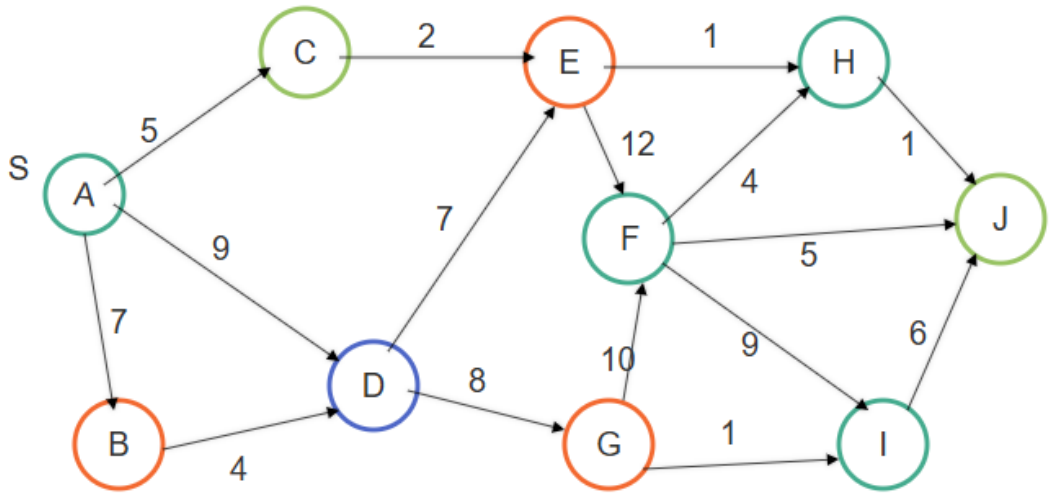
- **Vertices** represent intersections.
- **Edges** represent roads, each with a **travel time** (weight).
  The task is to calculate the **shortest travel time** from a **starting intersection** to **all other intersections** using **Dijkstra's Algorithm**.

Rules:

- Represent the city using a **weighted directed graph** structure.
- Each road can have a different travel time (weight).
- No negative edge weights are allowed.

Requirements:

- Build a graph structure with intersections (nodes) and roads (edges).
- Implement **Dijkstra's algorithm** to compute the shortest travel times from a source.
- Output the minimum travel time from the source to each intersection.



CSE207 – Data Structures

Project: Warehouse Robot Pathfinding

Project Description:

Simulate a robot navigating a warehouse represented as a **2D grid**.
The warehouse consists of:

- **Walls** (obstacles/shelves): cannot pass through.
- **Open paths**: the robot can move freely.
- **Packages**: must be collected.

The goal is to **collect all packages in the minimum number of steps**, using **Breadth-First Search (BFS)** to find the shortest path between locations.

Rules:

- The robot can move **up**, **down**, **left**, and **right** (no diagonal moves).
- Must use **BFS** to find the shortest path to each package.

Requirements:

- Represent the warehouse as a **2D matrix**:
  - 0 = wall (cannot move)
  - 1 = open path
  - P = package to collect
  - S = robot start position
- Calculate and display the minimum steps to collect all packages.
- Show the **movement path** and **order of package collection**.
-

```
[
  ['S', 1, 0, 'P'],
  [0,   1, 1,  1 ],
  ['P', 0, 1,  'P'],
  [1,  1, 1,   1 ]
]
```

CSE207 – Data Structures

Project: Virus Containment in a Network

Project Description:

Simulate a virus outbreak spreading across a network of computers.
The network is represented as a **graph** where:

- **Nodes** = Computers
- **Edges** = Direct connections between computers

When a virus infects a node, it **immediately spreads** to all directly connected nodes unless they are shut down.
The goal is to **identify the minimum number of computers to shut down** to **contain** the virus from spreading across the entire network.

Rules:

- The virus spreads to all **directly connected nodes**.
- Use **Breadth-First Search (BFS)** or **Depth-First Search (DFS)** to simulate the spread.
- Once a node is shut down (isolated), no further infection can pass through it.

Requirements:

- Represent the network using an **adjacency list** or **adjacency matrix**.
- Simulate the virus spread starting from an infected node.
- Determine the **minimum set of nodes to shut down** to block further spread.
- Display:
    - The nodes that need to be isolated.
    - The final state of the network after containment.